



JC4002 Knowledge Representation

Day 6: Classical Planning

Dr Ferdian Jovan

ferdian.jovan@abdn.ac.uk

Dr Aiden Durrant

aiden.durrant@abdn.ac.uk

Dr Jiangtian Nie

jiangtian.nie@abdn.ac.uk

Day 6

Classical Planning

1. Situation Calculus
2. STRIPS
3. ADL
4. Forward Search and Backward Search

Change Representation

- ❑ Sentences of first order logic are either true or false in an interpretation and stay that way, e.g.: `boy(john)`, `student(john)`, ...
- ❑ However, in reality, things can change through actions or overtime.
 - Change in our beliefs about the world.
From “Earth is flat” to “Earth is round”
 - Beliefs are about a changing world.
Initially `¬student(john)`, but later on, `student(john)`
 - ✓ Different from default reasoning in which might have initially believed `¬student(john)` and now we believe `student(john)`;
 - ✓ Here, we believe in both `¬student(john)` and `student(john)` but at different points in time.
- ❑ Situation Calculus is the earliest method in AI for representing actions.

Situation Calculus

The **situation calculus** is a dialect of FOL for representing dynamically changing worlds in which all changes are the result of **named actions**.

There are two distinguished sorts of terms:

❑ **Actions**, such as:

- ❑ $put(x, y)$; put object x on top of object y .
- ❑ $walk(loc)$; walk to location loc .
- ❑ $pickup(r, x)$; robot r picks up object x .

❑ **Situations**, denoting possible world histories. A distinguished constant S_0 and function symbol “ do ” are used.

- ❑ S_0 the initial situation, before any actions have been performed,
- ❑ $do(a, s)$ the situation that results from doing action a in situation s (i.e. **effect**).

❑ E.G.: $do(put(A, B), do(put(B, C), S_0))$ A situation that results from putting A on B after putting B on C in the initial situation

Fluents

- ❑ Predicates or functions whose values may vary from situation to situation are called **fluents**.
- ❑ These are written using predicate or function symbols whose last argument is a situation
 - ❑ E.g.: $holding(r, x, s)$, robot r is holding object x in situation s , can have:
 $\neg holding(r, x, s) \wedge holding(r, x, do(pickup(r, x), s))$
the robot is not holding the object x in situation s , but is holding it in the situation that results from picking it up.
- ❑ Note: there is no distinguished “current” situation. A sentence can talk about many different situations, past, present, or future.
- ❑ A precondition for an action is described by predicate symbol $Poss(a, s)$ to state that a may be performed in situation s
 - ❑ E.g.: $Poss(pickup(r, x), S_0)$ It is possible for the robot r to pickup object x in the initial situation

Actions in Situation Calculus

- ❑ It is necessary to include in a KB not only facts about the initial situation, but also about world dynamics: what the actions do.
- ❑ Actions are specified by first-order formulas
 - A precondition formula
$$\forall x, s. Poss(pickup(x), s) \equiv \forall z. \neg holding(z, s) \wedge \neg heavy(x) \wedge nextTo(x, s)$$
object x is possible to be picked up in situation s iff the object is not heavy, and is close to the agent, and the agent is not holding anything.
 - Effect axioms
$$\forall x, s. holding(x, do(pickup(x), s))$$
The agent is holding x when an action $pickup$ is performed at situation s .
$$\forall x, l_1, l_2, s. fuel(x, do(drive(x, l_1, l_2), s)) = fuel(x, s) - \frac{distance(l_1, l_2)}{mpg(x)}$$
What does this translate to in English?

The Frame Problem

- ❑ To really know how the world works, it is also necessary to know what fluents are *unaffected* by performing an action.
- ❑ These are sometimes called **frame axioms**.
 - ✓ $\forall x, c, s. Colour(x, c, s) \equiv \forall r. Colour(x, c, do(drop(r, x), s))$
Dropping an object does not change its color.
 - ✓ $\forall x, s. Poss(pickup(x), s) \equiv \forall z. \neg holding(z, s) \wedge \neg heavy(x) \wedge nextTo(x, s)$
- ❑ **Problem:** need to know a vast number of such axioms. (Few actions affect the value of a given fluent; **most leave it invariant**).
 - ✓ An object's colour is unaffected by picking things up, opening a door, using the phone, turning on a light, electing a new Prime Minister, etc.
- ❑ The frame problem:
 - ✓ In building KB, need to think of these $\sim 2 \times \text{Axioms} \times \text{Facts}$ about what does not change;
 - ✓ The system needs to reason efficiently with them.

Solution to Frame Problem

By making an

- ❑ Explanation Closure Assumption (fluents do not change value from situation to situation unless an action occurred that affected them); and
- ❑ Action Closure Assumption (the described set of actions are the only actions and these actions have no other effects than those described).

It is possible to automatically convert the set of effect axioms into *successor state axioms*, one for every fluent.

These are axioms of the form

- ❑ $\forall x_1, \dots, x_n, s. P(x_1, \dots, x_n, do(a, s)) \equiv \gamma(x_1, \dots, x_n, s)$
- ❑ $\forall x_1, \dots, x_n, s. f(x_1, \dots, x_n, do(a, s)) = y \equiv \gamma(x_1, \dots, x_n, y, s)$

Where γ is a formula that mentions the *previous situation* only (and the arguments of the fluent).

- ❑ $\forall x, s. holding(x, do(a, s)) \equiv holding(x, s) \wedge a \neq putdown(x) \wedge a = pickup(x)$

Projection Task

- ❑ What can we do with the situation calculus?
- ❑ We will see later that it can be used for planning.
- ❑ A simpler job we can handle directly is called the **projection task**.
 - ✓ Given a sequence of actions, determine what would be true in the situation that results from performing that sequence.
- ❑ This can be formalised as follows:
 - ✓ Suppose that $R(s)$ is a formula with a free situation variable s .
 - ✓ To find out if $R(s)$ would be true after performing $\langle a_1, \dots, a_n \rangle$ in the initial situation, we determine whether or not
$$KB \models R(do(a_n, do(a_{n-1}, \dots, do(a_1, S_0) \dots)))$$
- ❑ For example, using the effect and frame axioms from before, it follows that $\neg holding(toy, s)$ would hold after an agent did the sequence $\langle pickup(ball), pickup(square) \rangle$

Legality Task

- The projection task above asks if a condition would hold after performing a sequence of actions, but not whether that sequence can in fact be properly executed.
- We call a situation **legal** if it is the initial situation or the result of performing an action whose preconditions are satisfied starting in a legal situation.
- The **legality task** is the task of determining whether a sequence of actions leads to a legal situation.
- This can be formalized as follows:

To find out if the sequence $\langle a_1, \dots, a_n \rangle$ can be legally performed in the initial situation, we determine whether or not

$$KB \models Poss(a_i, do(a_{i-1}, \dots, do(a_1, S_0) \dots))$$

for every i such that $1 \leq i \leq n$.

Limitation: primitive actions

As yet, we have no way of handling in the situation calculus complex actions made up of other actions such as

- ☐ conditionals: If the car is in the driveway then drive else walk;
- ☐ iterations: while there is a block on the table, remove one;
- ☐ nondeterministic choice: pickup up some block and put it on the floor and others

Would like to define such actions in terms of the primitive actions, and inherit their solution to the frame problem.

Need a compositional treatment of the frame problem for complex actions.

Results in a novel programming language for discrete event simulation and high-level robot control

The Do Formula

For each complex action A , it is possible to define a formula of the situation calculus, $Do(A, s, s')$, that says that action A when started in situation s may legally terminate in situation s' .

- ❑ Primitive actions: $Do(A, s, s') \equiv Poss(A, s) \wedge s' = do(A, s)$
- ❑ Sequence: $Do([A; B], s, s') \equiv \exists s''. Do(A, s, s'') \wedge Do(B, s'', s')$
- ❑ Conditionals: $Do([if \phi then A else B], s, s') \equiv \phi(s) \wedge Do(A, s, s') \vee \neg\phi(s) \wedge Do(B, s, s')$
- ❑ Nondeterministic branch: $Do([A|B], s, s') \equiv Do(A, s, s') \vee Do(B, s, s')$
- ❑ Nondeterministic choice: $Do([\pi x. A], s, s') \equiv \exists x. Do(A, s, s')$

Note: programming language constructs with a purely logical situation calculus interpretation

Using the situation calculus

□ The situation calculus can be used to represent what is known about the current state of the world and the available actions.

□ The planning problem can then be formulated as follows:

□ Given a formula $Goal(s)$, find a sequence of actions $\mathbf{a}$ such that

$$KB \models Goal(do(\mathbf{a}, S_0)) \wedge Legal(do(\mathbf{a}, S_0))$$

where $do(\langle a_1, \dots, a_n \rangle, S_0)$ is an abbreviation for

$$do(a_n, do(a_{n-1}, \dots, do(a_2, do(a_1, S_0)) \dots))$$

and where $Legal(\langle a_1, \dots, a_n \rangle, S_0)$ is an abbreviation for

$$Poss(a_1, S_0) \wedge Poss(a_2, do(a_1, S_0)) \wedge \dots \wedge Poss(a_n, do(\langle a_1, \dots, a_{n-1} \rangle, S_0))$$

□ So: given a goal formula, we want a sequence of actions such that

✓ the goal formula holds in the situation that results from executing the actions, and

✓ it is possible to execute each action in the appropriate situation

Planning by answer extraction

- Having formulated planning in this way, we can use *Resolution* with answer extraction to find a sequence of actions:

$$KB \models \exists \mathbf{a}. Goal(do(\mathbf{a}, S_0)) \wedge Legal(do(\mathbf{a}, S_0))$$

with $\mathbf{a} = \langle a_1, \dots, a_n \rangle$

- The textbook has a detailed example on how it can be done
- However, “modern” classical planning algorithms use techniques that are customised to planning

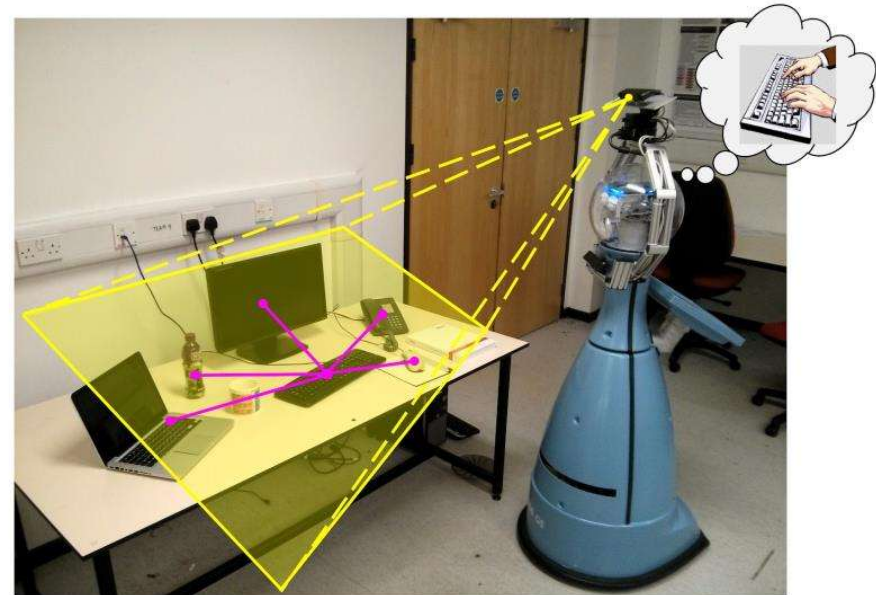
Day 6

Classical Planning

1. Situation Calculus
- 2. STRIPS**
3. ADL
4. Forward Search and Backward Search

STRIPS

- ❑ **Stanford Research Institute Problem Solver** (by Fikes & Nilsson, 1971).
- ❑ STRIPS is the simplest and the second oldest representation of operators in AI.
- ❑ Term used generally to refer to classical planning formulations.
- ❑ Original STRIPS planner performed an incomplete form of backward chaining.



The STRIPS representation

- ❑ STRIPS is an alternative representation to the pure situation calculus for planning.
 - from work on a robot called Shaky at SRI International in the 60's.
- ❑ In STRIPS, we do not represent histories of the world, as in the situation calculus.
- ❑ Instead, we deal with a single world state at a time, represented by a database of ground atomic wffs (e.g. *In(robot, room_1)*),
 - This is like the database of facts (ABox) used in Description Logic.
- ❑ Similarly, we do not represent actions as part of the world model (cannot reason about them directly), as in the situation calculus.
- ❑ Instead, actions are represented by **operators** that syntactically transform world models
 - An operator takes a DB and transforms it to a new DB

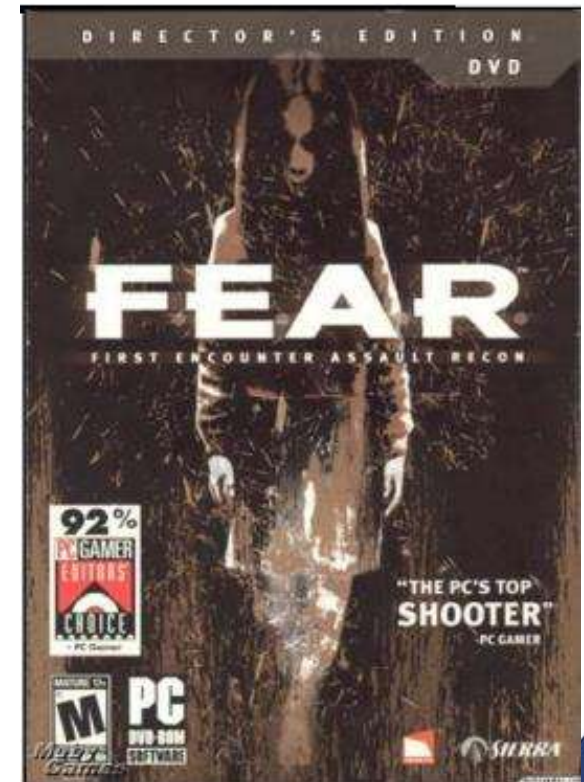
STRIPS Planning Problem

- ❑ Modern STRIPS Planning is covered by PDDL (Planning Domain Definition Language) – A more expressive planning language.
- ❑ Assumptions:
 - ❑ Complete knowledge captured in the initial state,
 - ❑ Actions are deterministic
- ❑ Given a planning domain **D** a classic planning problem
 - ❑ Inputs: The initial state, the goal, **D**
 - ❑ State Space
 - All states reachable by applying a sequence of actions to the initial state
 - **Actions** are operators with their parameters instantiated by constants mentioned in the initial state.
 - ❑ Output
 - A sequence of actions that transform the initial state to a state satisfying the goal

STRIPS Computational Aspects

- ❑ Even without variables, STRIPS planning is NP-hard.
- ❑ Many methods have been proposed to avoid redundant search through
 - Partial-order planners, macro operators, heuristics, graph plan, etc.
- ❑ However, early success of STRIPS shows how capable STRIPS can be.
- ❑ STRIPS was the planning engine for a Game called F.E.A.R (First Encounter Assault Recon) developed by Monolith Productions
 - ❑ Won Gamespot's Best AI Award in 2005

<https://www.gamespot.com/reviews/fear-review/1900-6135744/>



STRIPS Input: (Initial) State

- ❑ A state is represented as a conjunction of positive literals
- ❑ We can consider propositional literals (e.g. Rich $\wedge$ Happy)
- ❑ Typically, we will use first order literals for the input language, e.g.
At(Plane1, Melbourne) $\wedge$ At(Plane2, Sydney)
- ❑ Within a state description, such first order literals must be ground and function free
- ❑ We use:
 - ✓ closed world assumption – any condition not mentioned is false;
 - ✓ unique names assumption – distinct constants are not equal.
- ❑ In PDDL format, initial state is written as

(PROPOSITIONAL_LITERAL) or (PREDICATE ?x)

E.g.: (handempty) (clear ?x) (ontable ?x ?y)

STRIPS Input: Goal State

- ❑ Goals are partially specified states, represented as conjunctions of **positive ground** literals
 - Rich $\wedge$ Famous
 - `(:goal (and (rich) (famous)))` – PDDL format
- ❑ A state s satisfies a goal if s contains all the atoms in the goal (and possibly others).
 - ❑ Rich $\wedge$ Famous $\wedge$ Miserable
 - ❑ Rich $\wedge$ Famous $\wedge$ Happy
- ❑ Other possibilities
 - ❑ A more complex condition on a state, specified with a FOL formula.
 - ❑ A condition on the sequence of states visited by the plan (a “Temporally Extended Goal”)

STRIPS Input: Action Operators

- ❑ Operators specify the possible state transitions for a planning **domain**.
 - ❑ For any particular planning problem, it is necessary to use the initial state as well as the operator specification to determine the transitions possible for this particular problem (the actions).
 - ❑ An action is specified in terms of
 - ❑ The **preconditions** that must hold before it can be executed
 - ❑ The **effects** that occur when it is executed
- Action(Fly(p, from, to)),*
PRECOND: At(p, from) $\wedge$ Plane(p) $\wedge$ Airport(from) $\wedge$ Airport(to),
EFFECT: $\neg$ At(p, from) $\wedge$ At(p, to),
- ❑ Positive literals are added to the state, negative literals are removed.
 - ❑ Some planning systems use an **add** and **delete** lists.

Action Operator Semantics

- ❑ **Semantics** are described by how actions affect states.
- ❑ An action is **applicable** in any state that **satisfies** the precondition.
- ❑ For the first order action schema, determining applicability requires determining whether a legal substitution exists for variables in the precondition.
- ❑ Action: $Action(Fly(p, from, to))$,
 PRECOND: $At(p, from) \wedge Plane(p) \wedge Airport(from) \wedge Airport(to)$,
 EFFECT: $\neg At(p, from) \wedge At(p, to)$
- ❑ State:
 $At(P_1, JFK) \wedge Plane(P_1) \wedge Airport(JFK) \wedge Airport(SFO) \wedge Plane(P_2) \wedge At(P_2, SFO)$
- ❑ With substitution $\{p \setminus P_1, from \setminus JFK, to \setminus SFO\}$, action $Fly(P_1, JFK, SFO)$ is applicable

Action Operator Semantics

- ❑ Starting in a state s , the result of executing an applicable action a is a new state s' , which is identical except
 - ✓ Any **positive literal** P in the effects of a **is added** to s'
 - ✓ Any **negative literal** $\neg P$ in the effects of a **is removed** from s'
- ❑ States are sets; if a positive effect is already there, we **don't re-add** it; if a negative effect is **not present** we **ignore** it
- ❑ The STRIPS assumption: any literal not mentioned in the effect remains unchanged.—*STRIPS thus overcomes the frame problem.*
- ❑ A solution for a planning problem is an action sequence that, when executed in the initial state, results in a state that satisfies the goal.
- ❑ This is a simple form of the solution concept, there are more complex ones in non-classical planning.

Activity 1 – Find a plan for

In pairs, discuss the following question (5 minutes) and then share your answers with the rest of your colleagues (5 minutes)

$Init(At(C_1, SFO) \wedge At(C_2, JFK) \wedge At(P_1, SFO) \wedge At(P_2, JFK)$
 $\wedge Cargo(C_1) \wedge Cargo(C_2) \wedge Plane(P_1) \wedge Plane(P_2)$

$\wedge Airport(JFK) \wedge Airport(SFO))$

$Goal(At(C_1, JFK) \wedge At(C_2, SFO))$

$Action(Load(c, p, a),$

PRECOND: $At(c, a) \wedge At(p, a) \wedge Cargo(c) \wedge Plane(p) \wedge Airport(a)$

EFFECT: $\neg At(c, a) \wedge In(c, p)$

$Action(Unload(c, p, a),$

PRECOND: $In(c, p) \wedge At(p, a) \wedge Cargo(c) \wedge Plane(p) \wedge Airport(a)$

EFFECT: $At(c, a) \wedge \neg In(c, p)$

$Action(Fly(p, from, to),$

PRECOND: $At(p, from) \wedge Plane(p) \wedge Airport(from) \wedge Airport(to)$

EFFECT: $\neg At(p, from) \wedge At(p, to)$

How to Develop a Plan in STRIPS?

Use “means-ends analysis”

- ❑ If the agent is not at goal, then **measure the distance** (or difference) between the agent's current state and its goal state.
- ❑ Find an **operator** that the agent can take that will reduce this distance (difference), and select it if can bind the variables in preconditions to make preconditions true.
- ❑ Make the first false precondition of this operator the new “**subgoal**”, remembering old goal by pushing on stack.
- ❑ Recursively reduce difference by repeating above.
- ❑ When all preconditions for operator match, add the operator to the plan stack and update world model. Continue until we find an operator that can be executed in agent's current state
- ❑ When done, pop the actions off the stack to create the **plan** from start to goal.

How to Develop a Plan in STRIPS?

Early STRIPS algorithm (called gamma) adopt “means-end analysis” in combination with:

- ❑ Divide-and-Conquer forward search with islands
- ❑ Achieve one subgoal at a time : achieve a new goal literal without ever violating already achieved goal literals or maybe temporarily violating previous subgoals.
- ❑ Each subgoal is achieved via a matched rule, then its preconditions are subgoals and so on.

However, this recursive STRIPS algorithm can't deal with problems that can't be divided into subgoals that are dependent to one another.

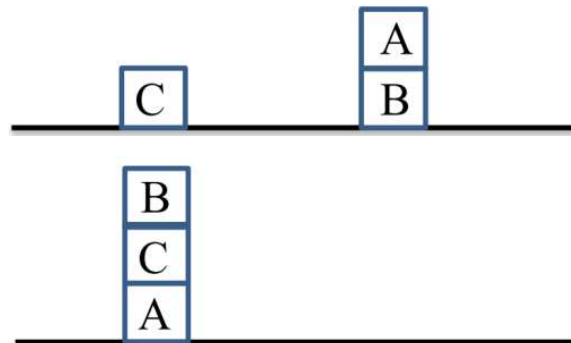
The Sussman Anomaly



RSTRIPS can't find a valid plan for the problem above due to two **dependent subgoals**.

Two possible orderings of subgoals:

✓ On(A,B) and On(B,C) or



✓ On(B,C) and On(A,B)

Non-interleaved planning does not work if goals are dependent

Day 6

Classical Planning

1. Situation Calculus
2. STRIPS
- 3. ADL**
4. Forward Search and Backward Search

Action Description Language (ADL)

In nutshell, ADL is a richer language than STRIPS. It allows for

- ❑ Positive and negative literals in states
- ❑ The open world assumption
- ❑ Quantified variables in goals as well as conjunctions and disjunctions
- ❑ Conditional effects
- ❑ (in)Equality predicates
- ❑ Types associated with variables

Like STRIPS, ADL originally had its own format / syntax (proposed by Pednault in 1987), but later it was integrated into more expressive planning language PDDL.

What is PDDL?

PDDL

- ❑ *PDDL = Planning Domain Definition Language standard encoding language for “classical” planning tasks*
- ❑ *Components of a PDDL planning task:*
 - *Objects: Things in the world that interest us.*
 - *Predicates: Properties of objects that we are interested in; can be true or false.*
 - *Initial state: The state of the world that we start in.*
 - *Goal specification: Things that we want to be true.*
 - *Actions/Operators: Ways of changing the state of the world.*
- ❑ *Syntax inspired by the Lisp programming language: e.g., prefix notation for formulas like (and (or (on A B) (on A C)))*
- ❑ *The planner input is separated into a **domain file** (predicates, types, action schemas) and a **problem file** (objects, initial state, goal).*

PDDL vs STRIPS

- ❑ A language that yields a search problem : actions translate into operators in search space.
- ❑ PDDL is a slight generalization of *STRIPS language*
- ❑ A state is
 - a set of positive ground literals (STRIPS)
 - a set of ground literals (PDDL)
- ❑ Closed world assumption : fluents that are not mentioned are false (STRIPS).
- ❑ If a literals is not mentioned, it is unknown (PDDL).
- ❑ The action schema consists of precondition and effect lists :
 - Only positive preconditions (STRIPS)
 - Positive or negative preconditions (PDDL)
- ❑ A set of action schemas is a definition of a planning domain.
- ❑ A specific problem is defined by an initial state (a set of ground atoms) and a goal: conjunction of atoms, some not grounded ($At(p, SFO)$, $Plane(p)$)

PDDL Domain File

```
(define (domain name)
  (:requirements :adl :typing)
  (:types type1 – object      type2 type3 – type1)

  (:predicates    <predicate1>
                 <predicate2> ... <predicaten>)

  (:action ...[DEFINED IN THE NEXT SLIDE])
)
```

PDDL Domain File

```
(define (domain name) name of domain
  (:requirements :adl :typing)
  (:types type1 – object type2 type3 – type1)

  (:predicates <predicate1>
               <predicate2> ... <predicaten>)

  (:action ...[DEFINED IN THE NEXT SLIDE])
)
```

PDDL Domain File

```
(define (domain name) name of domain
```

```
(:requirements :adl :typing) PDDL expressiveness options
```

```
(:types type1 – object type2 type3 – type1)
```

```
(:predicates <predicate1>  
<predicate2> ... <predicaten>)
```

```
(:action ...[DEFINED IN THE NEXT SLIDE])  
)
```

PDDL Domain File

```
(define (domain name) name of domain)
```

```
(:requirements :adl :typing) PDDL expressiveness options
```

```
(:types type1 – object type2 type3 – type1)
```

types of all available objects in this domain. Types are hierarchical and start with object as their root.

```
(:predicates <predicate1>  
             <predicate2> ... <predicaten>)
```

```
(:action ...[DEFINED IN THE NEXT SLIDE])
```

```
)
```

PDDL Domain File

(define (domain **name**) name of domain

(:requirements :adl :typing) PDDL expressiveness options

(:types type1 – object type2 type3 – type1)

types of all available objects in this **domain**. Types are hierarchical and start with object as their root.

(:predicates <predicate1> e.g. (on ?x - type2 ?y - type1)
 <predicate2> ... <predicaten>)

list of all **predicates** in this **domain** that are used in **action** definition.

(:action ...[DEFINED IN THE NEXT SLIDE])

)

PDDL Grammar: Action Schema

```
( :action name
  :parameters (?x – type1 ?y – type2 ?z – type3)

  :precondition    <predicate> (and <formula> ... <formula>)
                   (or <formula> ... <formula>) (not <formula>)
                   (forall (?x1 - type1 ... ?xn - typen) <formula>)
                   (exists (?x1 - type1 ... ?xn - typen) <formula>)

  :effect          <predicate> (not <predicate>)
                   (and <effect> ... <effect>) (when <formula> <effect>)
                   (forall (?x1 - type1 ... ?xn - typen) <effect>)

)
```

PDDL Grammar: Action Schema

```
( :action name action name
  :parameters (?x – type1 ?y – type2 ?z – type3)

  :precondition <predicate> (and <formula> ... <formula>)
                (or <formula> ... <formula>) (not <formula>)
                (forall (?x1 - type1 ... ?xn - typen) <formula>)
                (exists (?x1 - type1 ... ?xn - typen) <formula>)

  :effect <predicate> (not <predicate>)
            (and <effect> ... <effect>) (when <formula> <effect>)
            (forall (?x1 - type1 ... ?xn - typen) <effect>)

)
```

PDDL Grammar: Action Schema

(:action **name** **action name**

:parameters (**?x** – type1 **?y** – type2 **?z** – type3)

parameters listed in **:precondition** and **:effect**
parameters must be within type that has been
declared earlier in **:types**

:precondition <predicate> (and <formula> ... <formula>)
 (or <formula> ... <formula>) (not <formula>)
 (forall (?x1 - type1 ... ?xn - typen) <formula>)
 (exists (?x1 - type1 ... ?xn - typen) <formula>)

:effect <predicate> (not <predicate>)
 (and <effect> ... <effect>) (when <formula> <effect>)
 (forall (?x1 - type1 ... ?xn - typen) <effect>)

)

PDDL Grammar: Action Schema

(:action **name** **action name**

:parameters (?x – type1 ?y – type2 ?z – type3)

parameters listed in :precondition and :effect
parameters must be within type that has been
declared earlier in :types

:precondition

<predicate> (and <formula> ... <formula>)
(or <formula> ... <formula>) (not <formula>)
(forall (?x1 - type1 ... ?xn - typen) <formula>)
(exists (?x1 - type1 ... ?xn - typen) <formula>)
List of predicates that must hold in the current
state for the **action** to be applicable.
It can take any of the form above.

PDDL Grammar: Action Schema

(:action **name**
:parameters (?x – type1 ?y – type2 ?z – type3)
:precondition <predicate> (and <formula> ... <formula>)
(or <formula> ... <formula>) (not <formula>)
(forall (?x1 - type1 ... ?xn - typen) <formula>)
(exists (?x1 - type1 ... ?xn - typen) <formula>)
:effect
<predicate> (not <predicate>)
(and <effect> ... <effect>) (when <formula> <effect>)
(forall (?x1 - type1 ... ?xn - typen) <effect>)
The effect is a combination of **literals**, **conjunction**,
conditional effects, and **quantification over effects**
)

PDDL Problem File

```
(define (problem name)  
  (:domain name)  
  (:objects object1 – type1 object2 object3 – type2)  
  
  (:init <predicate1> <predicate2> ... <predicaten>)  
  
  (:goal <formula>)  
  
)
```

PDDL Problem File

```
(define (problem name) name of problem
  (:domain name)
  (:objects object1 – type1 object2 object3 – type2)

  (:init <predicate1> <predicate2> ... <predicaten>)

  (:goal <formula>)

)
```

PDDL Problem File

```
(define (problem name) name of problem  
  (:domain name) to which this problem belongs to  
  (:objects object1 – type1 object2 object3 – type2)  
  
  (:init <predicate1> <predicate2> ... <predicaten>)  
  
  (:goal <formula>)  
  
)
```

PDDL Problem File

(define (problem **name**) **name of problem**

(:domain **name**) to which this problem belongs to

(:objects **object1** – type1 **object2** **object3** – type2)

actual **objects** (or **constants** in FOL) which belongs to
types that are defined in the **domain** file.

(:init <predicate1> <predicate2> ... <predicaten>)

(:goal <**formula**>)

)

PDDL Problem File

(define (problem **name**) **name of problem**

(:domain **name**) to which this problem belongs to

(:objects **object1** – type1 **object2 object3** – type2)

actual **objects** (or **constants** in FOL) which belongs to
types that are defined in the **domain** file.

(:init <predicate1> <predicate2> ... <predicaten>)

list of all **ground predicates** that are true in the initial state

(:goal **<formula>**)

)

PDDL Problem File

(define (problem **name**) **name of problem**

(:domain **name**) to which this problem belongs to

(:objects **object1** – type1 **object2** **object3** – type2)

actual **objects** (or **constants** in FOL) which belongs to
types that are defined in the **domain** file.

(:init <predicate1> <predicate2> ... <predicaten>)

list of all **ground predicates** that are true in the initial state

(:goal <**formula**>)

a **formula** like in action **preconditions**

)

PDDL Domain File – Blocksworld

```
(define (domain Blocksworld)
  (:requirements :adl :typing)
  (:types block - object blueblock smallblock - block)
  (:predicates (on ?x - smallblock ?y - block)
               (ontable ?x - block) (clear ?x - block))
  (:action fromtable
    :parameters (?x - smallblock ?y - block)
    :precondition (and (not (= ?x ?y)) (clear ?x)
                      (ontable ?x) (clear ?y))
    :effect (and (not (ontable ?x)) (not (clear ?y)) (on ?x ?y))
  ))
```

PDDL Problem File – Blocksworld

```
(define (problem example)
  (:domain Blocksworld)
  (:objects a b c – smallblock  d e – block  f - blueblock)
  (:init (clear a) (clear b) (clear c) (clear d) (clear e)
         (clear f) (ontable a) (ontable b) (ontable c)
         (ontable d) (ontable e) (ontable f))
  )
  (:goal (and (on a d) (on b e) (on c f)))
)
```

Example Run of Planning Domain

Web Planner and Online Planning Domains

- ❑ <https://stripsfiddle.meteorapp.com/>
- ❑ <http://editor.planning.domains>

Generated Plan:

```
(move d1 d2 peg3)
(move d2 d3 peg2)
(move d1 peg3 d2)
(move d3 peg1 peg3)
(move d1 d2 peg1)
(move d2 peg2 d3)
(move d1 peg1 d2)
```

Action instance corresponding to first step:

```
(:action move
:parameters (d1 d2 peg3 )
:precondition (and (smaller peg3 d1) (on d1 d2)
                  (clear d1) (clear peg3)
)
:effect (and      (clear d2) (on d1 peg3)
                  (not (on d1 d2))
                  (not (clear peg3))
)
)
```

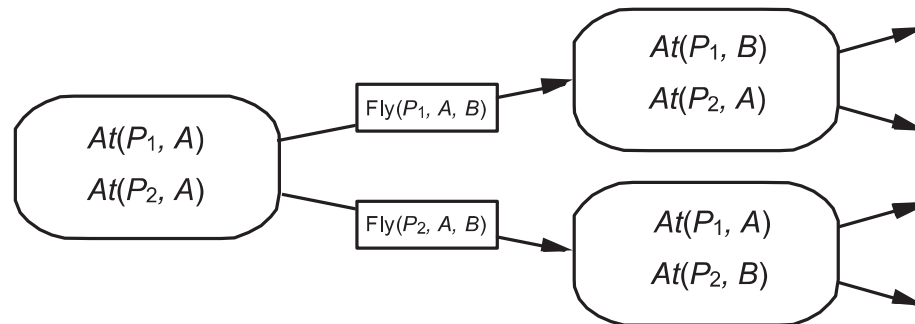
Day 6

Classical Planning

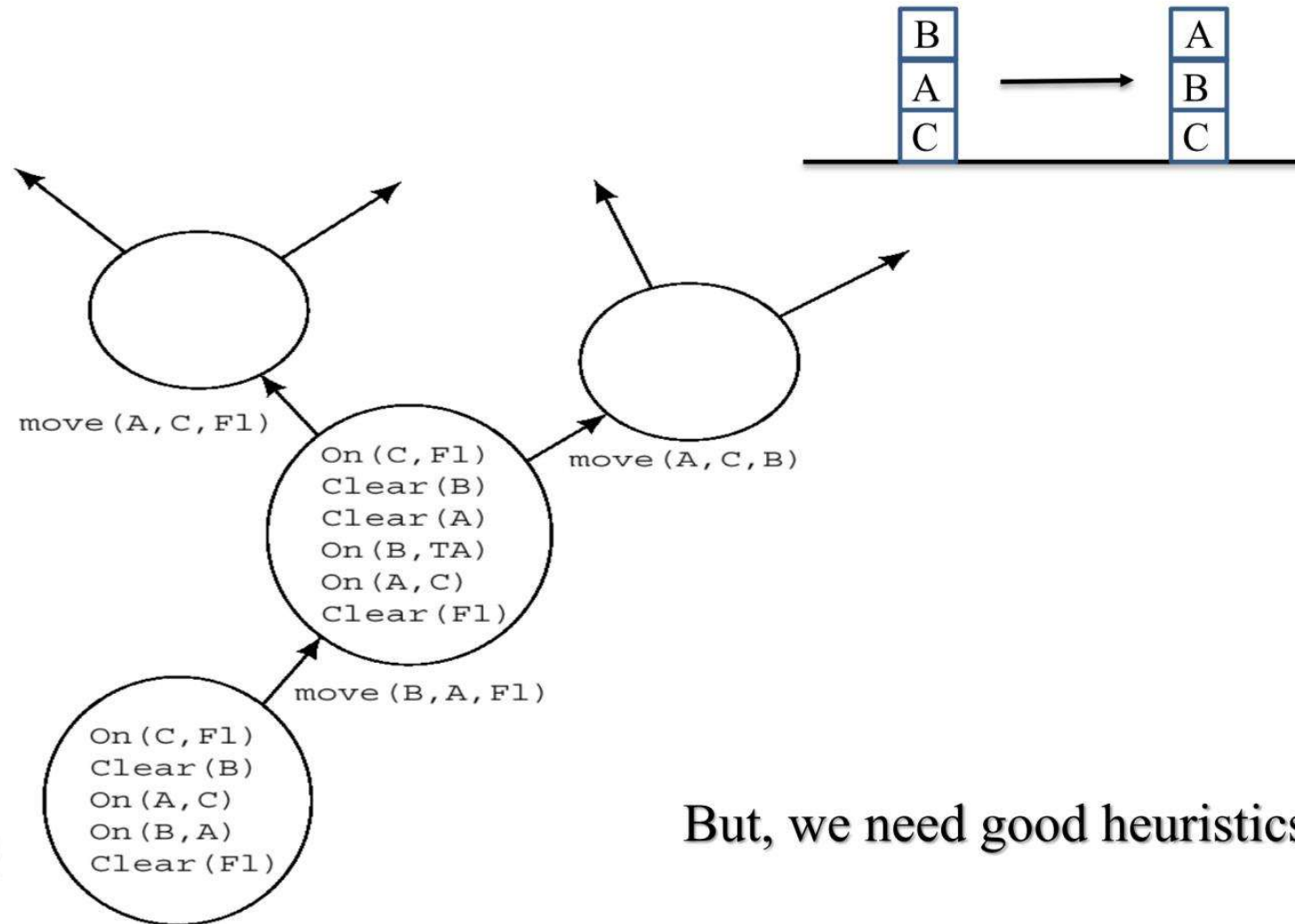
1. Situation Calculus
2. STRIPS
3. ADL
4. **Forward Search and Backward Search**

Forward Search

- ❑ **Progression** planning is similar to the approaches seen in search
- ❑ Start with initial state, consider sequences of actions until we find a sequence that reaches the goal state
- ❑ Specifying this type of planning problem requires
 - The initial state
 - The possible actions
 - The goal test
 - The step cost (typically 1 per action)



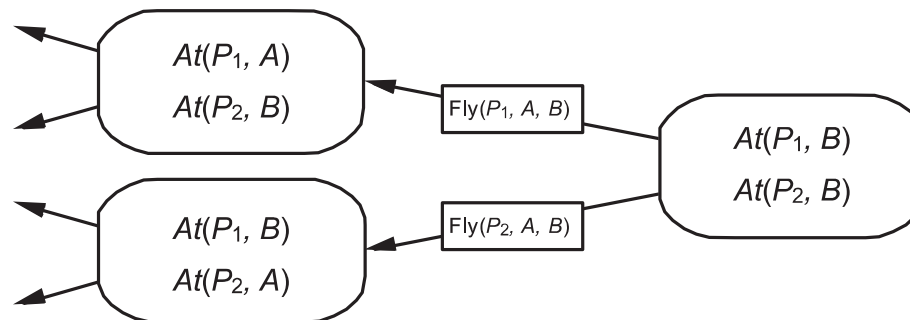
Forward Search Methods: can use A* with some h and g



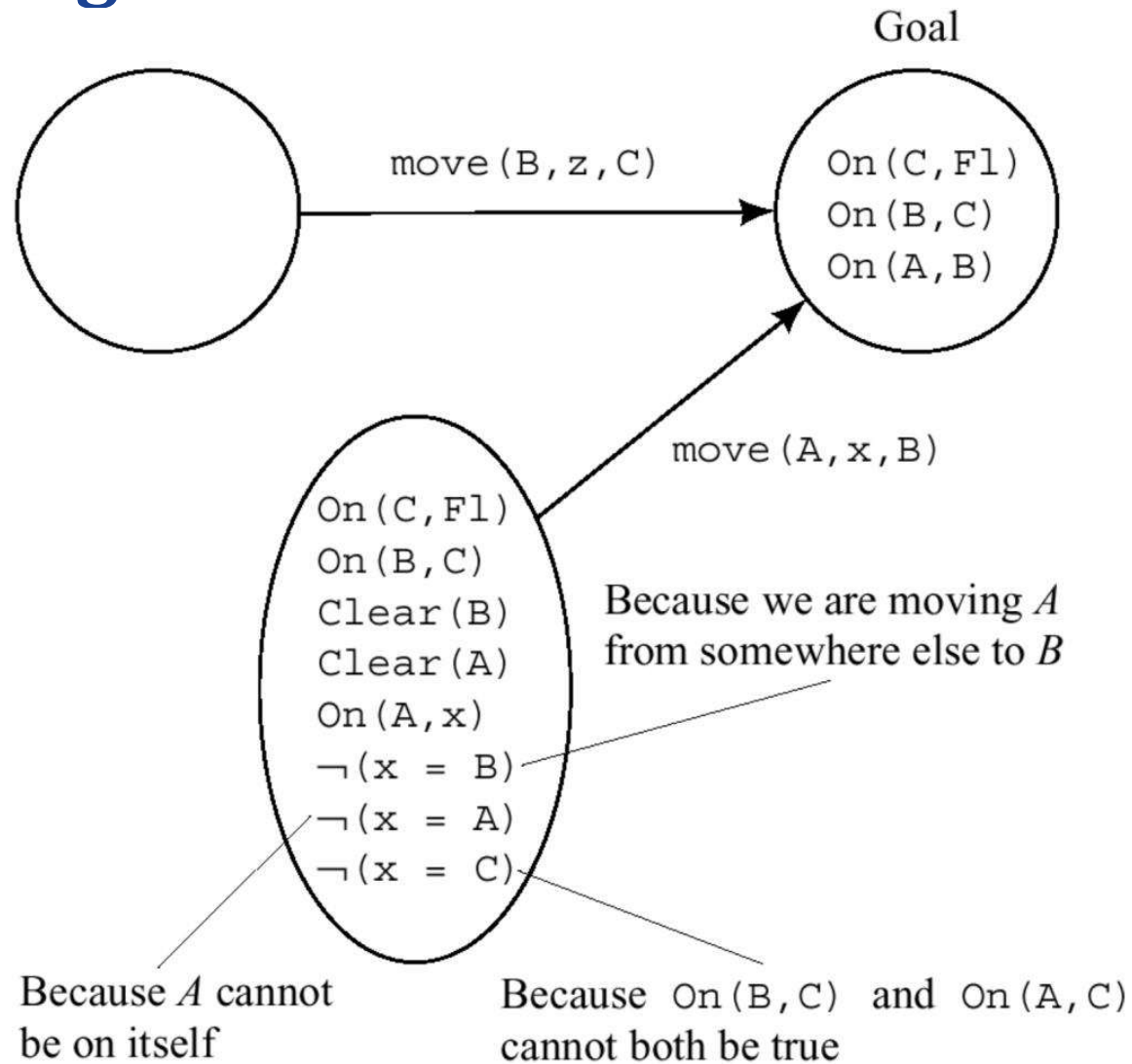
But, we need good heuristics

Backward Search

- ❑ Regression planning moves from goals to initial states
- ❑ With STRIPS, it is easy to generate the possible predecessors of a set of goal states
 - ✓ Consider only relevant actions, i.e. actions that achieve one of the conjuncts of a goal
- ❑ Any existing solution can be found by backwards search allowing only relevant actions
- ❑ Example: Air cargo transportation with 10 airports, 5 planes (per airport) and 20 pieces of cargo (per airport)
 - ✓ Thousands of actions leading out of the initial state
 - ✓ Only 20 actions working back from the goal.



Backward Search Methods: Regressing Action Schema



Forward vs Backward Search

- ❑ Forward search space nodes correspond to individual (grounded) states of the plan state-space.
- ❑ Backward search space nodes correspond to sets of plan state-space states, due to un-instantiated variables
 - ❑ because of this, designing good heuristics is hard(er)
 - ❑ however, it has smaller branching factor than Forward Search
- ❑ Forward search only feasible if good heuristics available

Summary

- Planning Problem Representation
- STRIPS
- ADL / PDDL
- Situation Calculus
- Forward and Backward Search